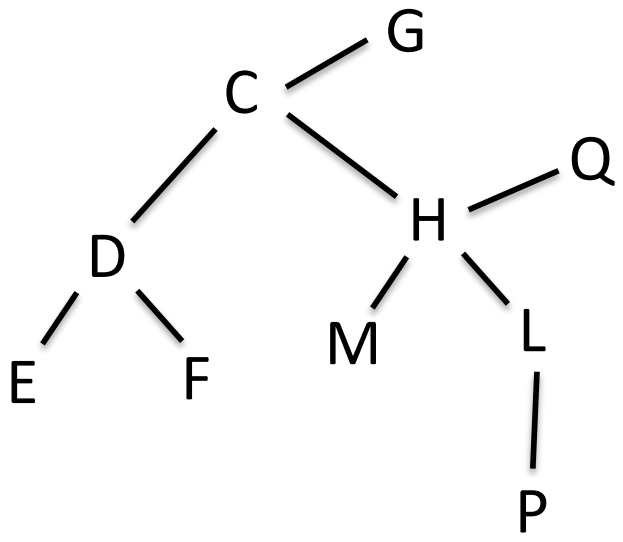
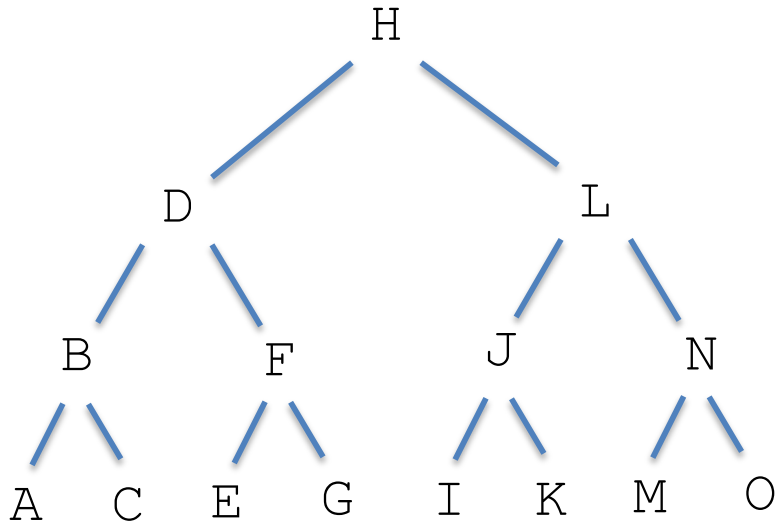
Informatica per le Biotecnologie
Algoritmica Lezione 8

Gli alberi



Un albero è un grafo
connesso aciclico

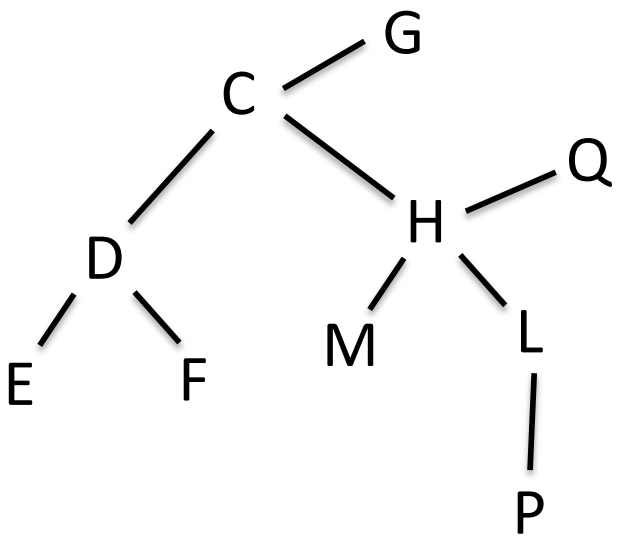
Se contiene n nodi, contiene n-1 archi
(si dimostra immediatamente per induzione)



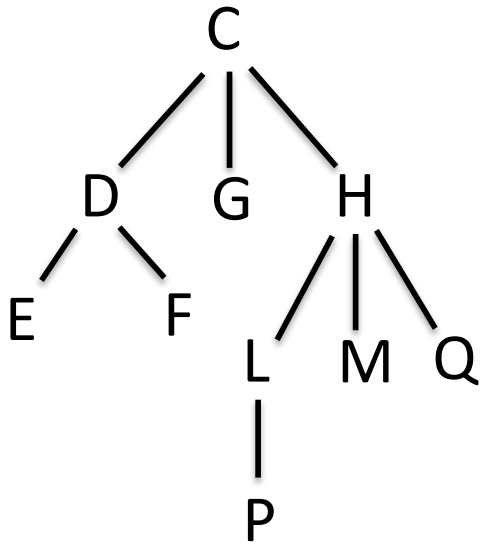
Albero binario completo

$n = 2^h - 1 = 15$ $h = 4$

con questo ordinamento delle etichette dei nodi vi si esegue la ricerca binaria che abbiamo già studiata in forma algoritmica



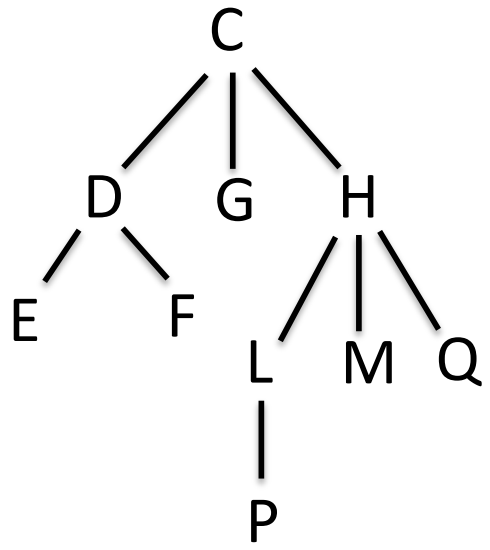
albero libero



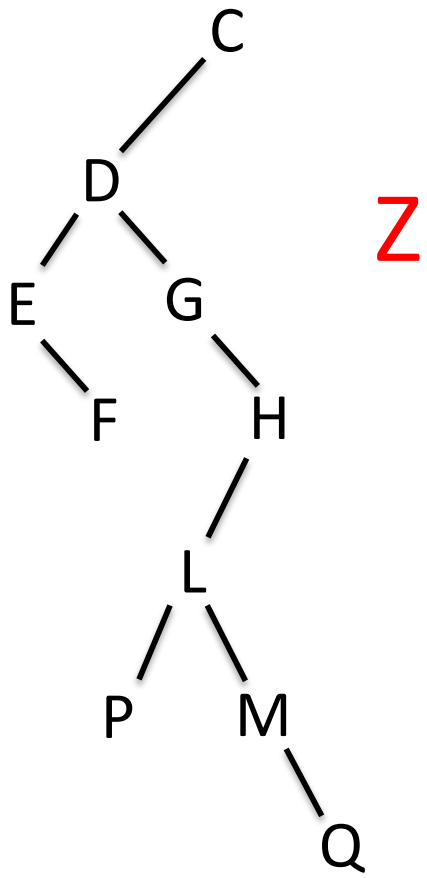
albero radicato

C è la radice. Se è definito un ordinamento tra i figli di ogni nodo, un albero radicato di dice ordinato

W



albero radicato W

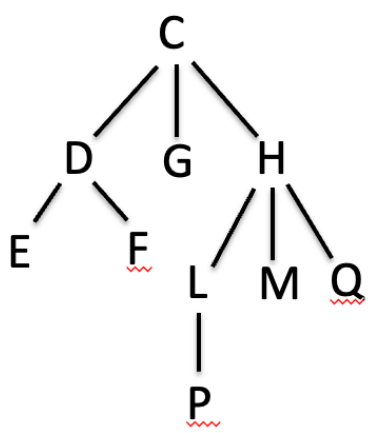


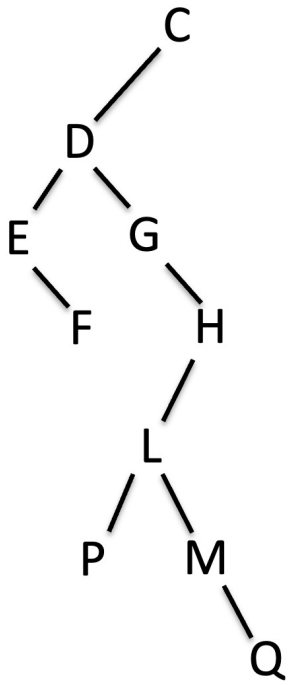
Z

albero binario
Immagine Z

In ogni nodo di Z, il figlio sinistro è il primo figlio in W e il figlio destro è il primo fratello in W

Un albero radicato trasformato in binario si rappresenta con un insieme di vettori





puntatore alla radice

R = 0

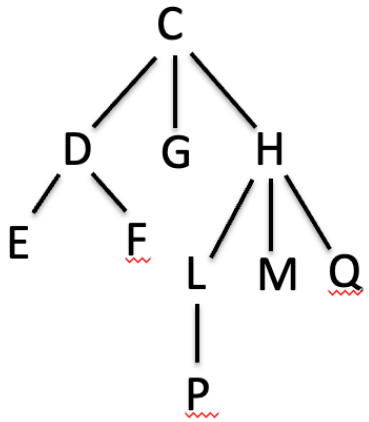
indice i	0	1	2	3	4	5	6	7	8	9
NODO	C	D	E	F	G	H	L	P	M	Q
FIGL	1	2	-1	-1	-1	6	7	-1	-1	-1
FRAT	-1	4	3	-1	5	-1	8	-1	9	-1

- 1 indica che il figlio o il fratello manca

vettore NODO: indicazione del PADRE

vettore ARCO: valore degli ARCHI

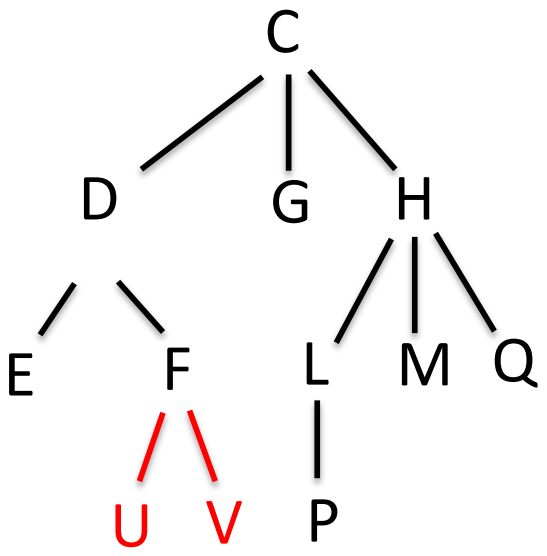
per esempio *cd* è il valore dell'arco C-D



R

i	0	1	2	3	4	5	6	7	8	9
NODO	C	D	E	F	G	H	L	P	M	Q
PADR	-1	0	1	1	0	0	5	6	5	5
ARCO	-1	*cd*	*de*	*df*	*cg*	*ch*	*hl*	*lp*	*hm*	*hq*

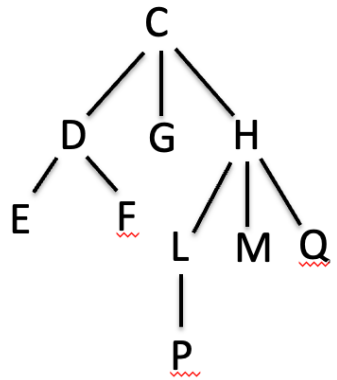
Trasformazioni di un albero
e impiego della lista libera LL



Inserzione di nodi

	R											LL		
i	0	1	2	3	4	5	6	7	8	9	10	11	12	13 ...
NODO	C	D	E	F	G	H	L	P	M	Q	U	V	*	* ...
FIGL	1	2	-1	10	-1	6	7	-1	-1	-1	-1	-1	*	* ...
FRAT	-1	4	3	-1	5	-1	8	-1	9	-1	11	-1	13	14 ...

VISITA DI UN ALBERO RADICATO Z



Z

Definizione ricorsiva di albero radicato:
un albero è un insieme di nodi tale che:

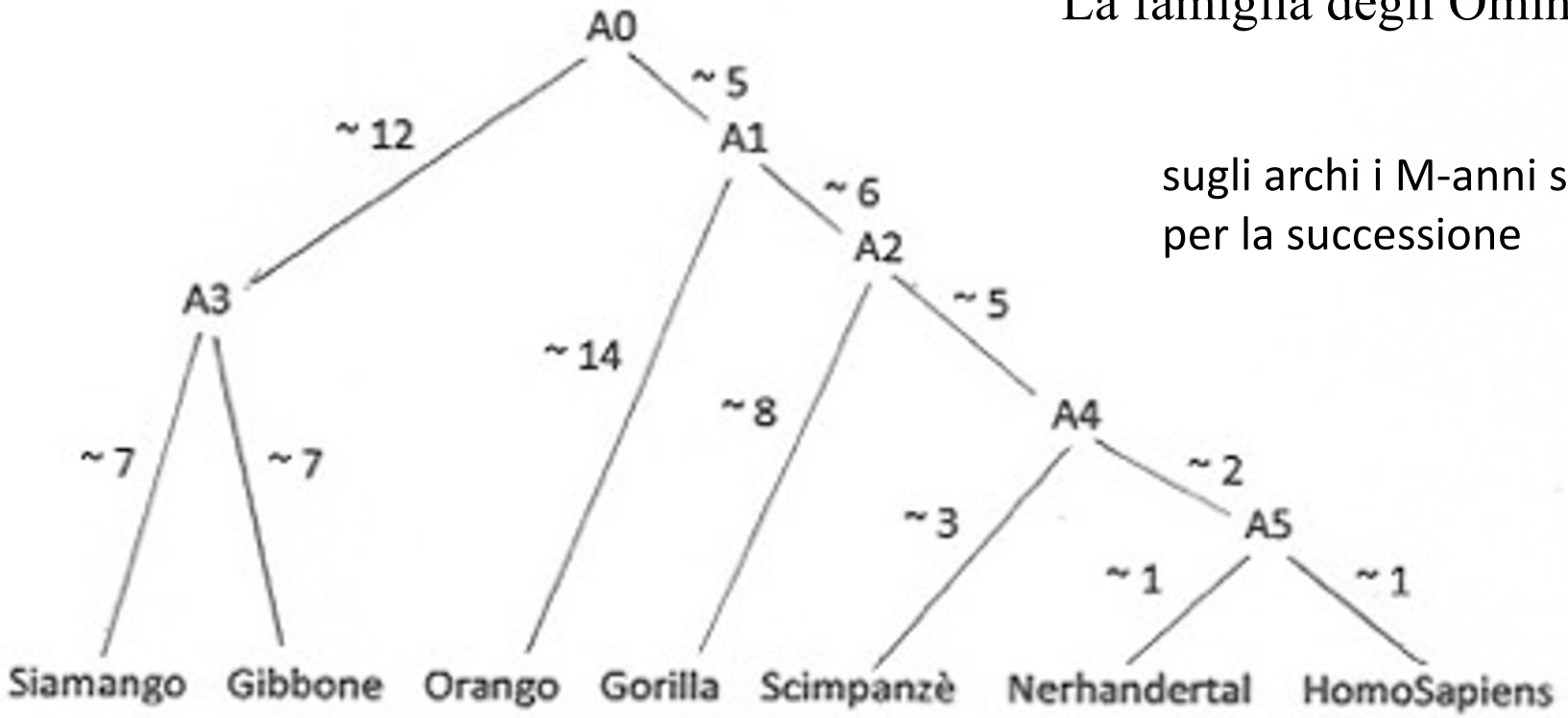
- un nodo di Z è destinato come radice;
- i rimanenti nodi di Z, se esistono, sono ripartiti in insiemi $Z_1, \ldots, Z_m$ ciascuno dei quali è un albero.

C D E F G H L P M Q

```
PREORDER(i)
// visita in preorder del sottoalbero di radice NODO(i)
   ESAMINA (NODO(i));
   if  (FIGL(i) ≠ -1) PREORDER(FIGL(I));
   if (FRAT(i) ≠ -1) PREORDER(FRAT(I));
```

CHIAMATA PREORDER (R)

La famiglia degli Ominoidi



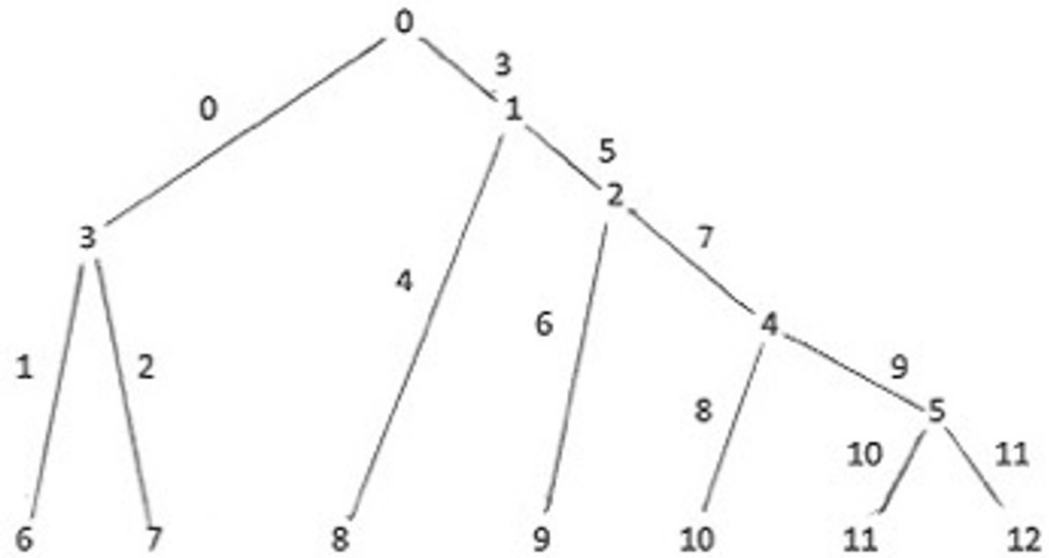
sugli archi i M-anni stimati per la successione

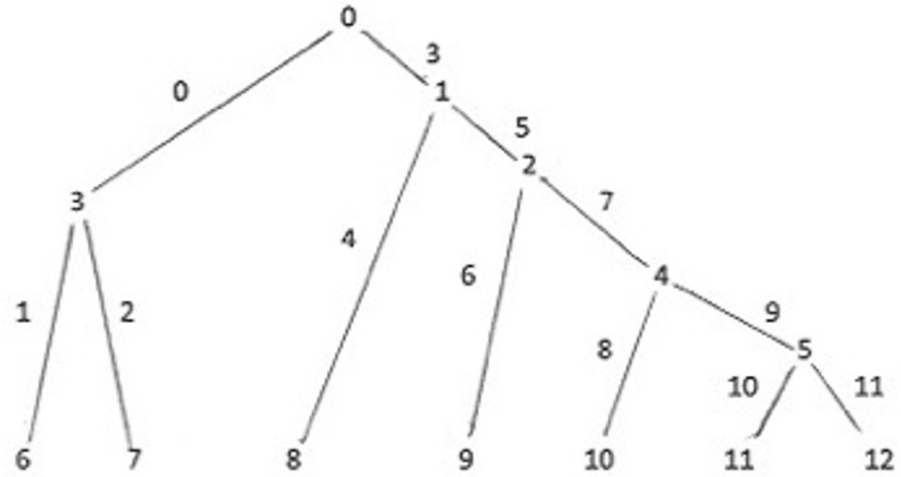
Ilobati

Ominidi

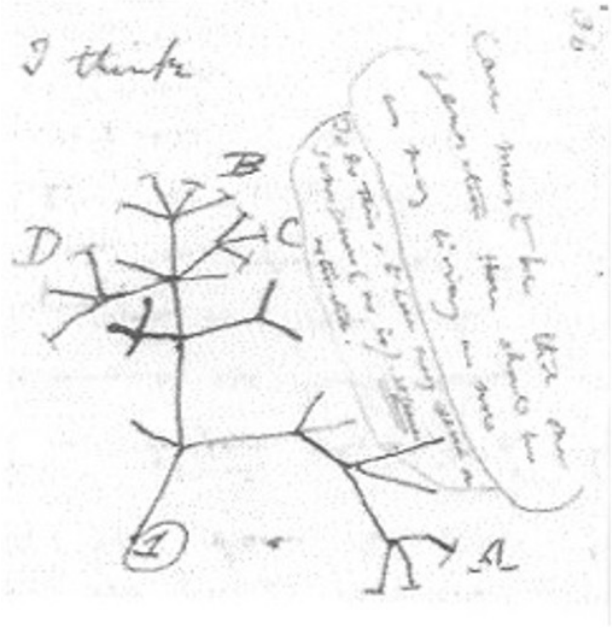
codice	0	1	2	3	4	5	6	7	8	9	10	11	12
nodo	A0	A1	A2	A3	A4	A5	Siam.	Gib.	Oran.	Gori.	Scimp.	Neh.	Homo

codice	0	1	2	3	4	5	6	7	8	9	10	11
arco	~12	~7	~7	~5	~14	~6	~8	~5	~3	~2	~1	~1





i	0	1	2	3	4	5	6	7	8	9	10	11	12
NODO	0	3	6	1	7	8	2	9	4	10	5	11	12
FIGL	1	2	-1	5	-1	-1	7	-1	9	-1	11	-1	-1
FRAT	-1	3	4	-1	-1	6	-1	8	-1	10	-1	12	-1
PADR	-1	0	1	0	1	3	3	6	6	7	8	10	10
ARCO	-1	0	1	3	2	4	5	6	7	8	9	10	11



Il primo albero filogenetico

Charles Darwin, 1873: l'*albero della vita*

Come si costruisce un albero filogenetico

Per un insieme di n organismi esistenti (A B C D E) e m caratteri (c1 c2 c3 c4 c5 c6), si costruisce una matrice di stato con organismi e caratteri rispettivamente associati alle righe e alle colonne.

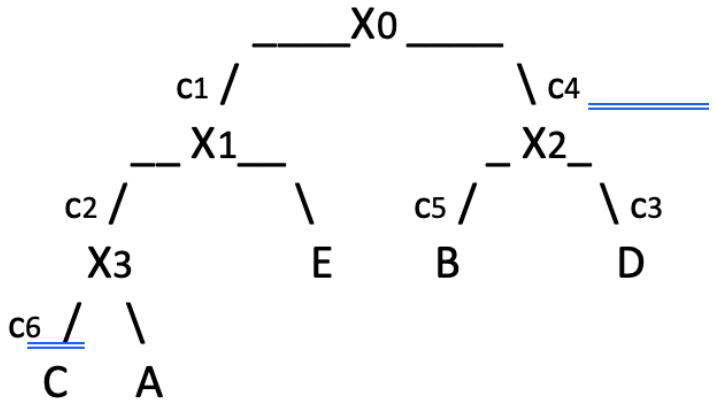
	c1	c2	c3	c4	c5	c6
A	1	1	0	0	0	0
B	0	0	0	1	1	0
C	1	1	0	0	0	1
D	0	0	1	1	0	0
E	1	0	0	0	0	0

Un esempio con caratteri binari:

0 indica il valore antico

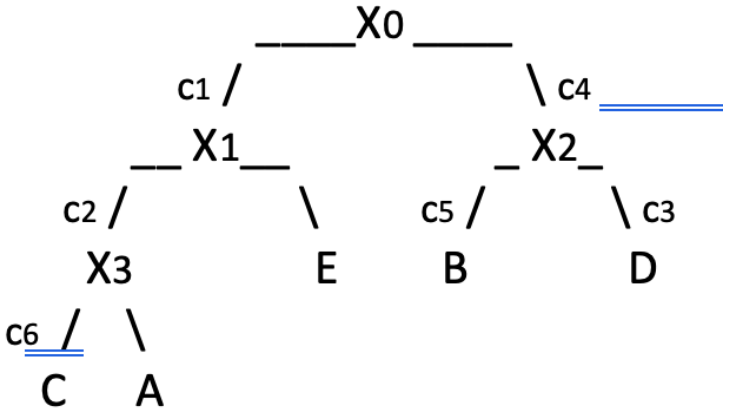
1 Indica il valore attuale mutato

	c1	c2	c3	c4	c5	c6
A	1	1	0	0	0	0
B	0	0	0	1	1	0
C	1	1	0	0	0	1
D	0	0	1	1	0	0
E	1	0	0	0	0	0



L’albero corrispondente: sugli archi sono indicati i caratteri che hanno generato la diramazione

Definizione. Una filogenia perfetta ammette un albero T tale che, per ogni carattere c e per ogni suo stato s, l'insieme di tutti i nodi di T per cui c è nello stato s forma un sottoalbero di T



	c1	c2	c3	c4	c5	c6
A	1	1	0	0	0	0
B	0	0	0	1	1	0
C	1	1	0	0	0	1
D	0	0	1	1	0	0
E	1	0	0	0	0	0

Per ogni colonna c_i indichiamo con U_i l'insieme di righe che contengono 1.

Per esempio per c_4 abbiamo $U_4 = \{B,D\}$

Lemma. Una matrice binaria corrisponde a una filogenia perfetta se e solo se, per ogni coppia di colonne c_i , c_j , gli insiemi U_i , U_j sono disgiunti o uno contiene l'altro.

Informatica per le Biotecnologie
Complessità computazionale

Esempio: somma di elementi di un insieme, certificati e problemi in NP

Problema $P_{\Sigma 2}$. **Dati un insieme S di n interi positivi e un intero k, stabilire se esistono due elementi di S la cui somma è k.**

	0	1	2	3	4	5	6	7
A	22	31	7	47	30	18	15	40

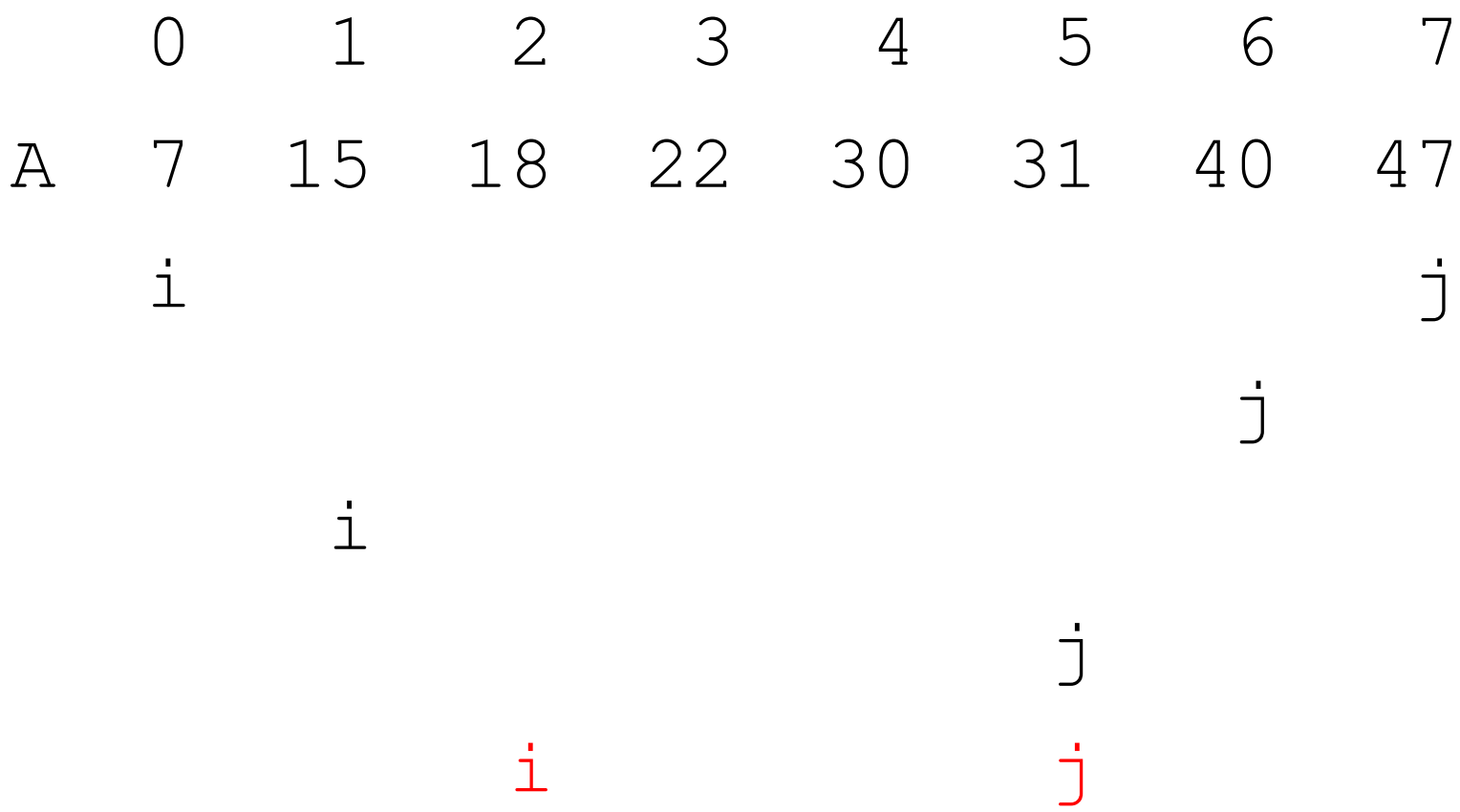
Per $k = 49$ risposta affermativa: $A[1] + A[5] = 49$.

Per $k = 42$ risposta negativa.

Possiamo provare con tutte le coppie di elementi

```
SIGMA2(A, k)
  for (i = 0; i ≤ n − 2; i++)
    for (j = i + 1; j ≤ n − 1; j++)
      if (A[i] + A[j] == k)
          return una coppia esiste;
  return nessuna coppia esiste;
```

Il tempo richiesto da SIGMA2 è di ordine $O(n^2)$, ma possiamo fare meglio ordinando l'insieme


0 1 2 3 4 5 6 7
A 7 15 18 22 30 31 40 47
i
j
j
i
j
i
j
A[2] + A[5] = 49

Operando in questo modo il tempo richiesto è $\theta(n \log n)$ per ordinare il vettore, più $O(n)$ per lo scorrimento di i e j .

In totale $\theta(n \log n)$

Limite inferiore al tempo di calcolo di $P_{\Sigma 2}$

Albero di decisione: Il numero di soluzioni è $S(n) = n(n-1)/2+1$ (1 rappresenta il caso in cui nessuna coppia abbia somma k). L'albero di decisione ha profondità $\geq \log_2 S(n) = \log_2(n^2/2 - n/2+1)$, quindi di ordine $\Omega(\log n)$.

Molto più forte è il limite $\Omega(n)$ che deriva dalla osservazione che tutti gli elementi di A devono essere esaminati.almeno una volta.

Rimane un gap tra i due limiti $\Omega(n)$ e $O(n \log n)$

Generalizzazione di $P_{\Sigma 2}$ a P_{Σ}:

dato un insieme S di n interi positivi e un intero k, stabilire se esiste un sottoinsieme di S di dimensioni arbitrarie la cui somma degli elementi è k.

Nell'esempio precedente la risposta affermativa per $k = 92$: $A[2] + A[4] + A[6] + A[7] = 92$.

Per P_Σ non si conosce un algoritmo polinomiale.

Si può affrontare associando ad A un *vettore di appartenenza* V contenente interi 0, 1 in cui $A[i]$ fa parte di una scelta di elementi se e solo se $V[i] = 1$.

	0	1	2	3	4	5	6	7
A	22	31	7	47	30	18	15	40
V	0	0	1	0	1	0	1	1

Vettore V per il sottoinsieme { 7, 30, 15, 40 }

Generiamo tutti i possibili vettori V di n elementi e verifichiamo se per uno di essi è verificata l'uguaglianza:

$$\sum_{i=0}^{n-1} A[i] \cdot V[i] = k$$

Esiste per esempio un algoritmo ricorsivo per generare tutti i vettori *V* . Vedi dispense

Tale algoritmo richiede tempo esponenziale perché questi vettori sono in numero 2^n.

Per moltissimi di problemi esponenziali un *certificato* K permette di verificare, attraverso un algoritmo polinomiale $VER(K, D)$, se i dati D dell'istanza considerata hanno la proprietà desiderata.

Nel problema P_{Σ} il vettore V relativo a una soluzione costituisce un certificato (il calcolo della relazione su k richiede tempo $\Theta(n)$).

Definizione. $\mathcal{NP}$ è l'insieme dei problemi decisionali verificabili in tempo polinomiale.

P_Σ appartiene dunque a $\mathcal{NP}$.

Notare che:

L'*esistenza* di un certificato K non ha alcun effetto sull'algoritmo di soluzione perché K non è noto a priori.

Per risolvere un problema in $\mathcal{NP}$ è necessario ricorrere a un metodo enumerativo che esegua un numero esponenziale di prove.

Per ogni problema in $\mathcal{P}$ si può verificare l'esistenza di una soluzione in tempo polinomiale ignorando il certifiato e risolvendo direttamente il problema. Da ciò segue che:fondamentale:

$$\mathcal{P} \subseteq \mathcal{NP}.$$

Non è mai stato dimostrato se vale il contenimento in senso stretto tra le due classi, cioè se sia $\mathcal{P} \neq \mathcal{NP}$. Tale quesito è tra i più importanti problemi irrisolti nella matematica.

Esempi di problemi in $\mathcal{NP}$

1. Biologia molecolare: dato un insieme arbitrario di sequenze di DNA stabilire se esiste una sottosequenza comune di lunghezza assegnata.

2. Manifattura: stabilire se un insieme di pezzi di lamiera possono essere ottenuti tagliandoli da un foglio assegnato disponendoli opportunamente su di esso.

3. Scheduling: dato un insieme di procedure da eseguire su un insieme di macchine, stabilire se tali procedure possono essere distribuite tra le macchine in modo che tutte queste terminino di operare entro un tempo prefissato.

4. **Algebra: stabilire se un'equazione algebrica di secondo grado in due variabili x, y, a coefficienti interi, ammette una soluzione in cui x e y hanno valori interi (se l'equazione è di primo grado il problema è in $\mathcal{P}$).**

Questi problemi si sanno risolvere solo provando un numero esponenziale di possibili soluzioni. Si può però verificare la correttezza di una soluzione in tempo polinomiale.